

實作步驟

▼ 實作步驟

▼ 第一階段：開發環境準備與工具配置

1. **安裝基礎環境**：確保電腦已安裝 [Node.js](#) 與 [npm](#)。準備好你熟悉的程式碼編輯器（如 VS Code）。
2. **初始化區塊鏈開發框架**：使用 **Hardhat** 來建立智能合約開發環境。這能幫助你在本地端編譯、測試與部署合約。

```
# 建立專案資料夾
mkdir whistleblower-dapp
cd whistleblower-dapp

# 初始化 Hardhat 開發環境（這是目前最主流的區塊鏈開發工具）
npx hardhat init

# 安裝必要的工具包（via npm）
npm install dotenv @openzeppelin/contracts ethers
```

3. 請在 `contracts/` 資料夾下建立一個新檔案 `Whistleblower.sol`，這是你系統的「心臟」。

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

contract Whistleblower
enum Status { Pending, UnderInvestigation, Resolved }

struct Case {
    uint256 id;
    string ipfsCID;    // 加密文件的地址
    string contentHash; // 文件雜湊值（SHA-256）
}
```

```

        Status status;
        uint256 timestamp;
    }

    mapping(uint256 => Case) public cases;
    uint256 public caseCount;

    event ReportSubmitted(uint256 indexed caseId, string ipf
sCID, uint256 timestamp);
    event StatusUpdated(uint256 indexed caseId, Status newSt
atus);

    // 匿名舉報功能
    function submitReport(string memory _ipfsCID, string mem
ory _contentHash) public {
        caseCount++;
        cases[caseCount] = Case(
            caseCount,
            _ipfsCID,
            _contentHash,
            Status.Pending,
            block.timestamp
        );
        emit ReportSubmitted(caseCount, _ipfsCID, block.time
stamp);
    }

```

```

# 編譯與本地
npx hardhat compile

# 顯示"Compiled 1 Solidity file successfully"表示成功

```

4. 啟動local測試區塊鏈

#啟動本地測試區塊鏈

`npx hardhat node`

#執行後，你會看到 20 個測試帳號 (Account #0 ~ #19) 和私鑰。請不要關閉這個視窗，讓它持續運行

(1) 在 `scripts/` 資料夾中，建立一個新檔案：`deploy.js`

```
import hre from "hardhat";

async function main() {
  const Whistleblower = await hre.ethers.getContractFactory("Whistleblower");

  console.log("正在部署合約...");

  // v5 的部署方式
  const contract = await Whistleblower.deploy();

  // 修改這裡：從 waitForDeployment() 改回 .deployed()
  await contract.deployed();

  console.log("=====");
  console.log("舉報系統合約部署成功！");
  console.log("合約地址:", contract.address); // v5 使用 .address 屬性
  console.log("=====");
}

main().catch((error) => {
  console.error(error);
  process.exitCode = 1;
});
```

(2) 另外開啟一個新的終端機視窗B（原本運行 `npx hardhat node` 的那個不要動 → 視窗A），進入專案路徑後輸入：

```
# 視窗B!!  
npx hardhat run scripts/deploy.js --network localhost
```

5. 建立前端專案 (DApp 介面)

現在我們要建立讓舉報者操作的網頁。我們使用 Vite + React，這比傳統方法快很多。

在 `whistleblower-dapp` 資料夾下執行：

```
# 建立名為 frontend 的資料夾  
npm create vite@latest frontend -- --template react  
  
# 進入前端資料夾  
cd frontend  
  
# 安裝前端必備工具 (ethers 用來跟錢包溝通)  
npm install  
npm install ethers  
  
# 啟動前端開發伺服器  
npm run dev
```

6. **準備區塊鏈錢包與測試幣**：安裝 MetaMask 擴充功能，並切換至測試網（如 Ethereum Sepolia 或 Polygon Amoy）。前往水龍頭 (Faucet) 領取測試代幣，作為後續部署合約的 Gas Fee。

(1) 看到 `http://localhost:5174/` 就代表網頁伺服器正在執行

(2) 準備 ABI 說明書

前端需要合約的「說明書」才能溝通。請去你的專案根目錄找到：

`artifacts/contracts/Whistleblower.sol/Whistleblower.json` 把這個檔案複製，貼到 `frontend/src/` 資料夾下。

(3) 修改 `frontend/src/App.jsx`

打開這個檔案，將裡面的內容全部刪除，換成下面這段實用的代碼（請記得把中間的 `YOUR_CONTRACT_ADDRESS` 換成你剛才部署成功的地址）：

```

import { useState } from 'react';
import { ethers } from 'ethers';
import whistleblowerArtifact from './Whistleblower.json'; // 載入說明書

const CONTRACT_ADDRESS = "把你的 0x 地址貼在這裡";

function App() {
  const [report, setReport] = useState("");
  const [status, setStatus] = useState("");

  async function requestAccount() {
    await window.ethereum.request({ method: 'eth_requestAccounts' });
  }

  async function sendReport() {
    if (!report) return;
    console.log("開始執行 sendReport...");

    if (typeof window.ethereum !== 'undefined') {
      try {
        await requestAccount();
        const provider = new ethers.providers.Web3Provider(window.ethereum);
        const signer = provider.getSigner();
        const contract = new ethers.Contract(CONTRACT_ADDRESS, whistleblowerArtifact.abi, signer);

        console.log("正在發送交易...");
        setStatus("請在 MetaMask 確認交易...");

        // *** 關鍵修改：傳入兩個參數 ***
        // 我們暫時把 report 文字同時填入 _ipfsCID 和 _content
        // Hash 位置

```

```

    const transaction = await contract.submitReport(re
port, "dummy_hash_for_test");

    console.log("交易已發送:", transaction.hash);
    setStatus("交易處理中...");

    await transaction.wait();
    setStatus("舉報成功！資料已上鏈。");
    setReport("");
  } catch (err) {
    console.error("出錯了:", err);
    // 這裡會幫你抓出具體原因
    setStatus("錯誤: " + (err.data?.message || err.mess
age));
  }
}

return (
  <div style={{ padding: "50px", textAlign: "center",
fontFamily: "sans-serif" }}>
    <h1>匿名舉報系統 (DApp)</h1>
    <textarea
      value={report}
      onChange={(e) => setReport(e.target.value)}
      placeholder="請輸入舉報內容..."
      style={{ width: "300px", height: "100px", paddin
g: "10px" }}
    />
    <br /><br />
    <button onClick={sendReport} style={{ padding: "10
px 20px", cursor: "pointer" }}>
      送出舉報 (MetaMask)
    </button>
    <p>{status}</p>
  </div>

```

```
);  
}  
  
export default App;
```

6. 如何測試？

(1) 打開瀏覽器，前往 <http://localhost:5174/>。

(2) 確保你的瀏覽器裝有 **MetaMask** 插件。

(3) **關鍵點**：你需要將 MetaMask 連接到你的「本地網路」。

- 在 MetaMask 點擊「新增網路」。
- RPC URL 輸入：<http://127.0.0.1:8545>。
- 鏈 ID 輸入：[31337](#)。
- 貨幣符號：[ETH](#)。

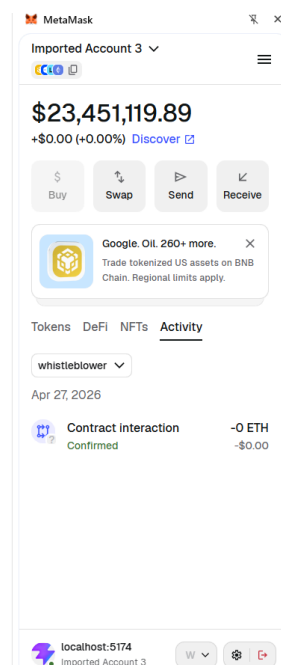
(4) 從視窗 A 的那 20 個地址中，隨便找一個**私鑰 (Private Key)** 匯入到 MetaMask，你就有用不完的測試幣了！

匿名舉報系統 (DApp)

請輸入舉報內容...

送出舉報 (MetaMask)

舉報成功！資料已上鏈。



(5) 視窗 A (Hardhat Node 終端機)

這是最即時的證據。當你在網頁點擊送出並在 MetaMask 確認後，視窗 A 會跳出新的日誌：

- **關鍵字：**找 `eth_sendRawTransaction`。
- **內容：**它會顯示 `Contract call: Whistleblower#submitReport`（或你的函數名稱）。
- **狀態：**如果看到 `Transaction: 0x...` 且沒有報錯訊息，就代表區塊鏈已經把這筆資料打包進區塊了。

▼ 第二階段：智能合約開發 (Contract Layer)

1. **撰寫 Solidity 核心合約：**定義一個 `Report` 結構 (Struct)，包含提交者地址、文件 Hash 值、IPFS CID、提交時間戳記與案件狀態（如：0=已提交、1=受理中、2=調查中、3=已結案）。
2. **實作核心功能：**撰寫 `submitReport(string memory _cid, string memory _hash)` 函數讓舉報者上傳資料；撰寫 `updateStatus(uint256 _reportId, uint8 _newStatus)` 函數供調查委員會更新進度。
3. **設計權限控制：**透過 `modifier` 限制只有「預先設定的調查委員會地址」才能呼叫狀態更新函數，確保案件狀態不可被一般人隨意竄改。
4. **部署至測試網：**撰寫部署腳本，將合約發布至 Polygon 測試網（推薦 Polygon，因其手續費較低）。部署完成後，記錄下合約地址 (Contract Address) 與應用二進制介面 (ABI)。

▼ 第三階段：前端加密與去中心化儲存 (Storage Layer)

1. **建立前端專案：**使用 React (搭配 Vite 或 Create React App) 或 Vue.js 初始化前端介面。
2. **實作本地端加密：**引入密碼學套件（如 `crypto-js`）。在舉報者選取文件後，直接在瀏覽器端使用對稱加密（如 AES）對文件進行加密。**務必確保明文資料絕不離開使用者的裝置。**
3. **生成雜湊存證 (Hash)：**將「加密後」的文件資料使用 SHA-256 演算法產生出一組唯一的 Hash 值，作為數位指紋。
4. **上傳至 IPFS：**透過 API 將加密後的文件上傳至 IPFS 網路，並取得系統回傳的內容識別碼 (CID)。

▼ 第四階段：Web3 錢包串接與上鏈整合 (Frontend DApp)

1. **連接區塊鏈錢包**：在前端引入 `ethers.js` 或 `wagmi` 套件。實作「Connect Wallet」按鈕，讓使用者連接 MetaMask。
2. **落實隱私保護提示**：在連接錢包的 UI 介面上，加入強烈的視覺提示，引導使用者「請創建並使用一個全新的、無交易紀錄的一次性錢包地址進行提交」，以確保物理上的身份隔離。
3. **呼叫智能合約上鏈**：將前面取得的 IPFS CID 與 SHA-256 Hash，透過 `ethers.js` 打包成區塊鏈交易，呼叫合約的 `submitReport` 函數。使用者在 MetaMask 確認並支付 Gas Fee 後，資料即永久上鏈。

▼ 第五階段：金鑰傳遞與調查委員會後台

1. **處理加密金鑰的傳遞**：由於文件已用 AES 加密，你需要設計金鑰傳遞機制。最安全的做法是在前端使用調查委員會公開的「非對稱加密公鑰 (RSA Public Key)」將 AES 密鑰加密後，一併上傳至 IPFS 或記錄在合約中。這樣只有擁有 RSA 私鑰的委員會能解開 AES 密鑰。
2. **建立調查委員會儀表板**：開發一個獨立的管理介面，讓委員可以連接特定錢包，讀取區塊鏈上的 CID，從 IPFS 下載加密文件，進行解密，並呼叫合約更新調查進度。